INFORMATION AND COMMUNICATION TECHNOLOGIES · FINAL EXAM

DormFix

A web app where dormitory students report broken equipment and staff update the repair status.

Tattimbek Yerkhan · Bagitova Mubina · ST-2606

Prototype: **https://shynstudy.lol/dormfix/**

1. Problem and target users

PROBLEM

Students need a simple way to report broken lights and other shared equipment in their dormitory. Reports get lost in chats or after talking to staff, and students do not know when the problem is fixed.

TARGET USERS

- **Students** send repair reports.
- **Administrators and workers** see the reports and update the repair status.

2. Project solution

DormFix is a web app connected to a Supabase database. It has 3 screens and 5 main actions:

FUNCTION	WHO USES IT	RESULT
Find reports in a dormitory	Staff	List of all reports in that building
Send a new report	Student	Report is saved with status Submitted
Update the status	Staff	Status changes, for example to Assigned
Delete a report	Staff	Wrong or duplicate report is removed
Show statistics	Staff	Number of reports for each category

Expected result: every report is stored in one place, and staff can see which repairs are needed most.

3. HCI diagram and user interaction

Find reports

Choose a dormitory → **Find**
See the list of reports
Each row: **Update** and **Delete**

⇄

New report

Choose student, dormitory, category
Type location and problem
Send report → “Report DF-… sent”

⇄

Statistics

Show statistics
Table: category → total reports

The tabs at the top move between the 3 screens. After every action the app shows a green message (success) or a red message (error).

Database and connection to Supabase

TABLES USED BY THE APP (FROM ASSIGNMENT 4)

report

report_id (PK)
student_id (FK)
dormitory_id (FK)
category_id (FK)
location_detail, description, status

user

user_id (PK)
full_name, role

dormitory

dormitory_id (PK)
name

category

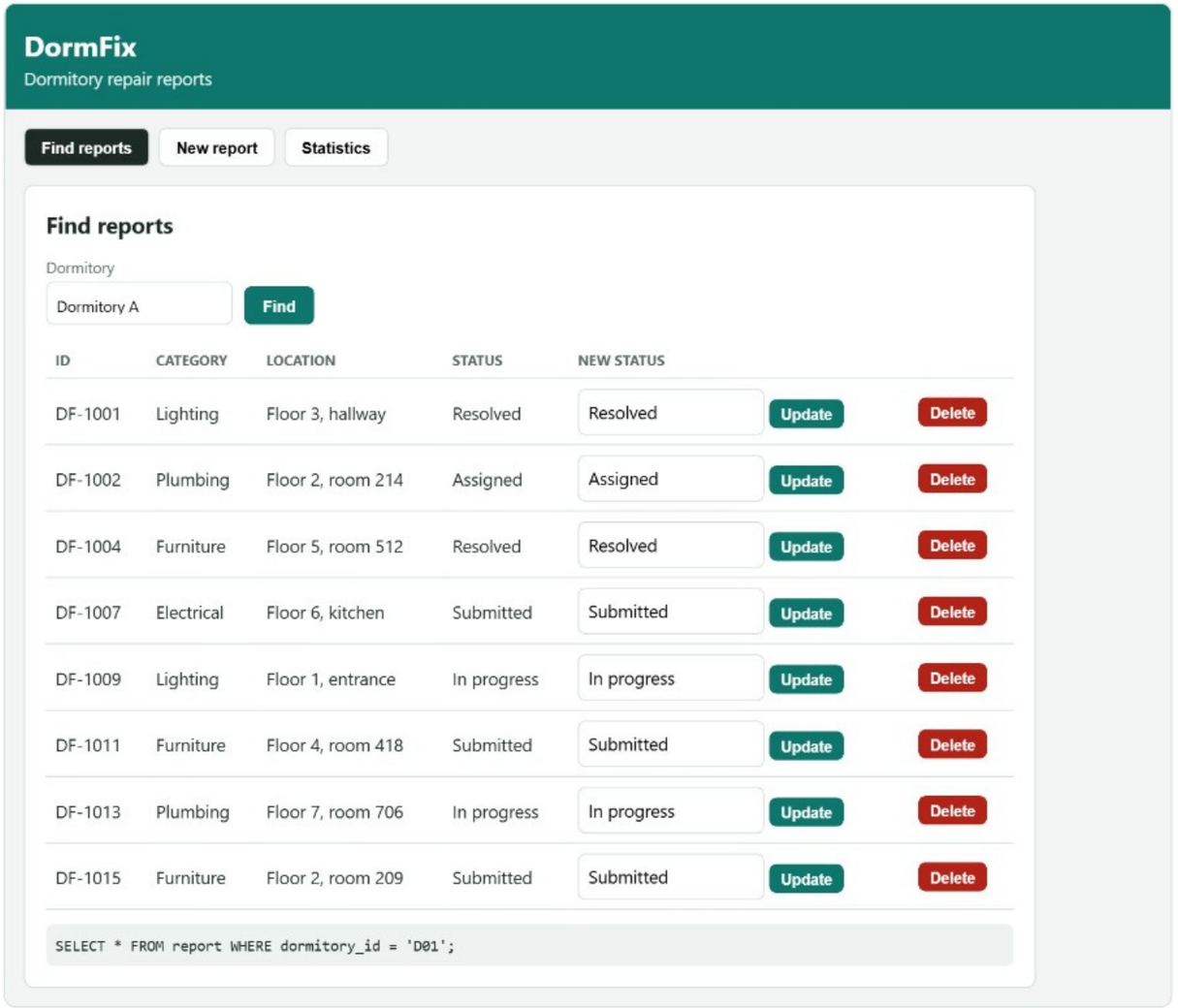
category_id (PK)
name

One student, dormitory or category has many reports.

HOW THE APP TALKS TO THE DATABASE

1. The user presses a button in the browser.
2. The app sends the request with the Supabase JavaScript client and the project's public key.
3. Supabase runs the SQL on the PostgreSQL database.
4. The result comes back and the app shows it on the screen.

4. Button “Find” → SELECT with WHERE



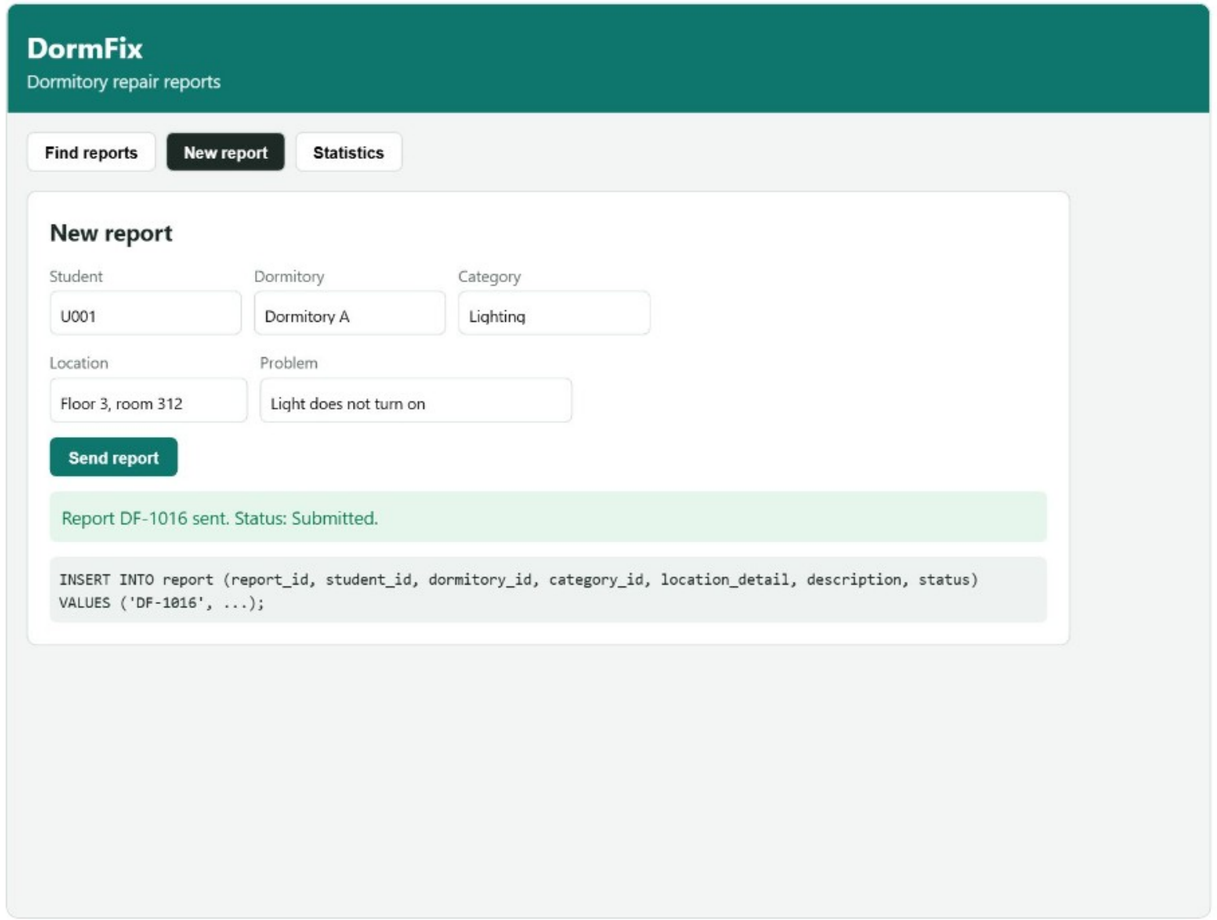

```
SELECT * FROM report
WHERE dormitory_id = 'D01';
```

Table: report

Result: 8 reports in Dormitory A.

User sees: the list of reports with ID, category, location and status.

Button “Send report” → INSERT



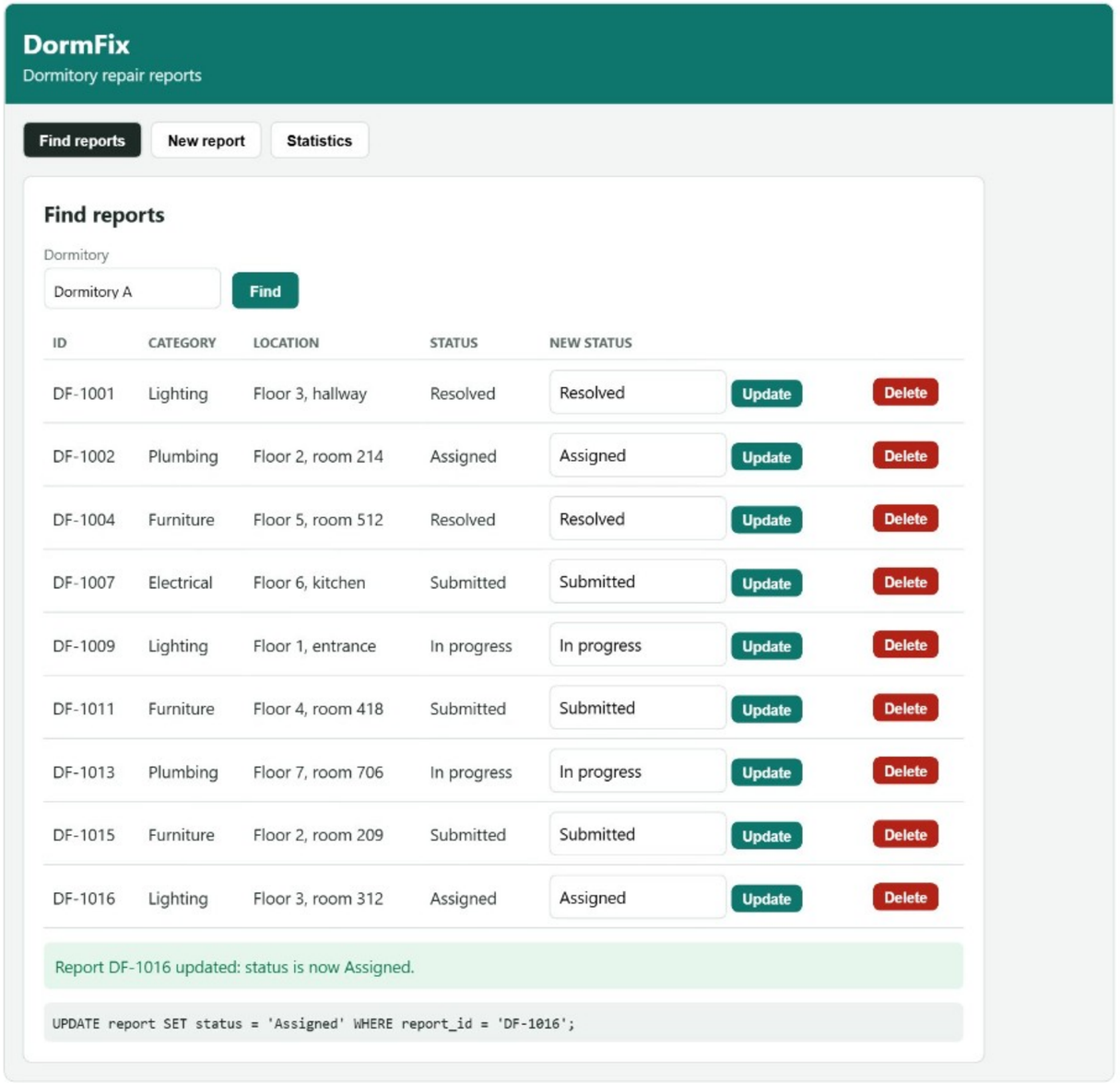

```
INSERT INTO report (report_id, student_id,
 dormitory_id, category_id,
 location_detail, description, status)
VALUES ('DF-1016', 'U001', 'D01', 'C01',
 'Floor 3, room 312',
 'Light does not turn on', 'Submitted');
```

Table: report

Result: a new row DF-1016 is created in Supabase.

User sees: “Report DF-1016 sent. Status: Submitted.”

Button “Update” → UPDATE



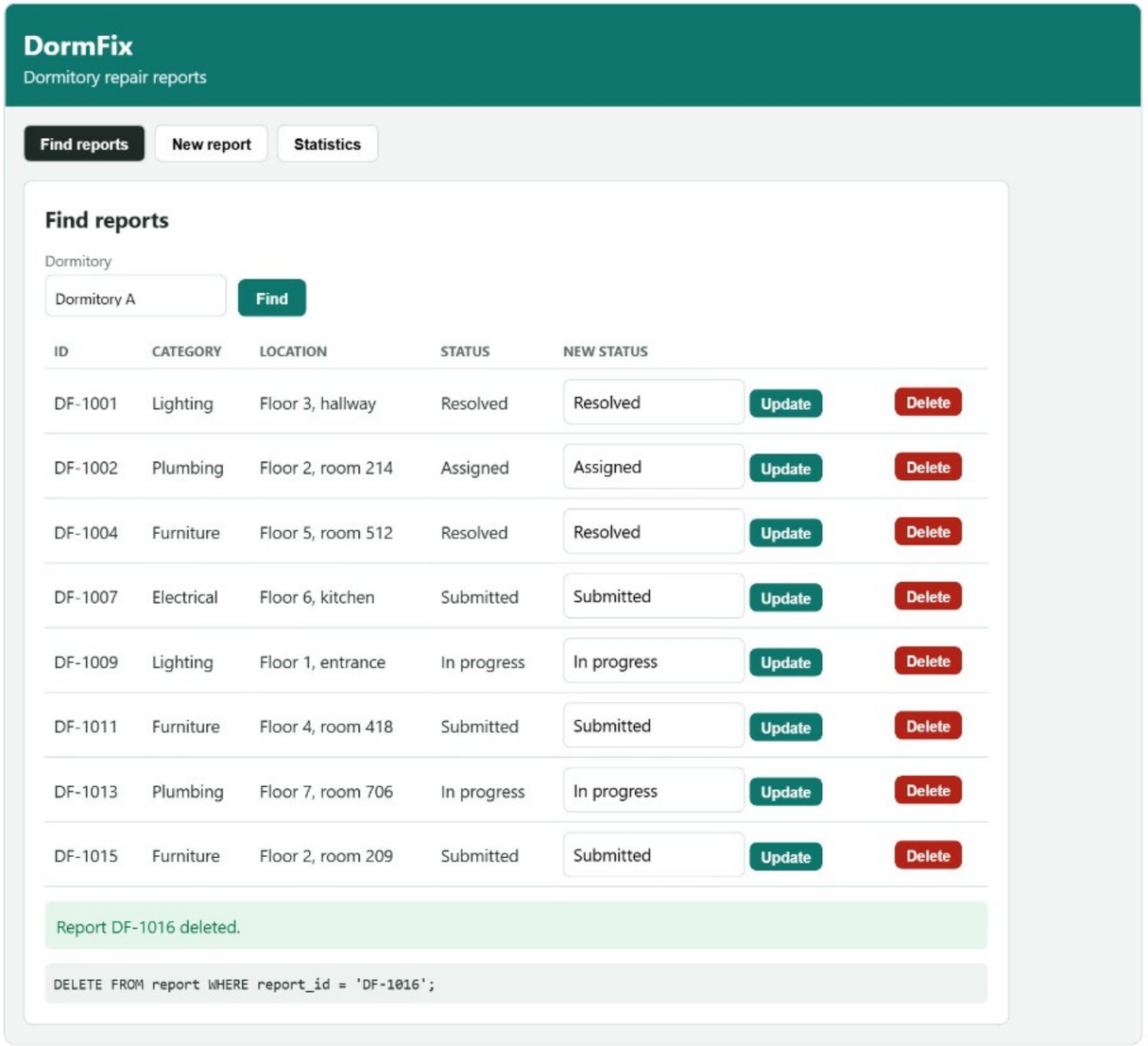

```
UPDATE report
SET status = 'Assigned'
WHERE report_id = 'DF-1016';
```

Table: report

Result: the status of DF-1016 changes from Submitted to Assigned.

User sees: “Report DF-1016 updated: status is now Assigned.” The list shows the new status.

Button “Delete” → DELETE



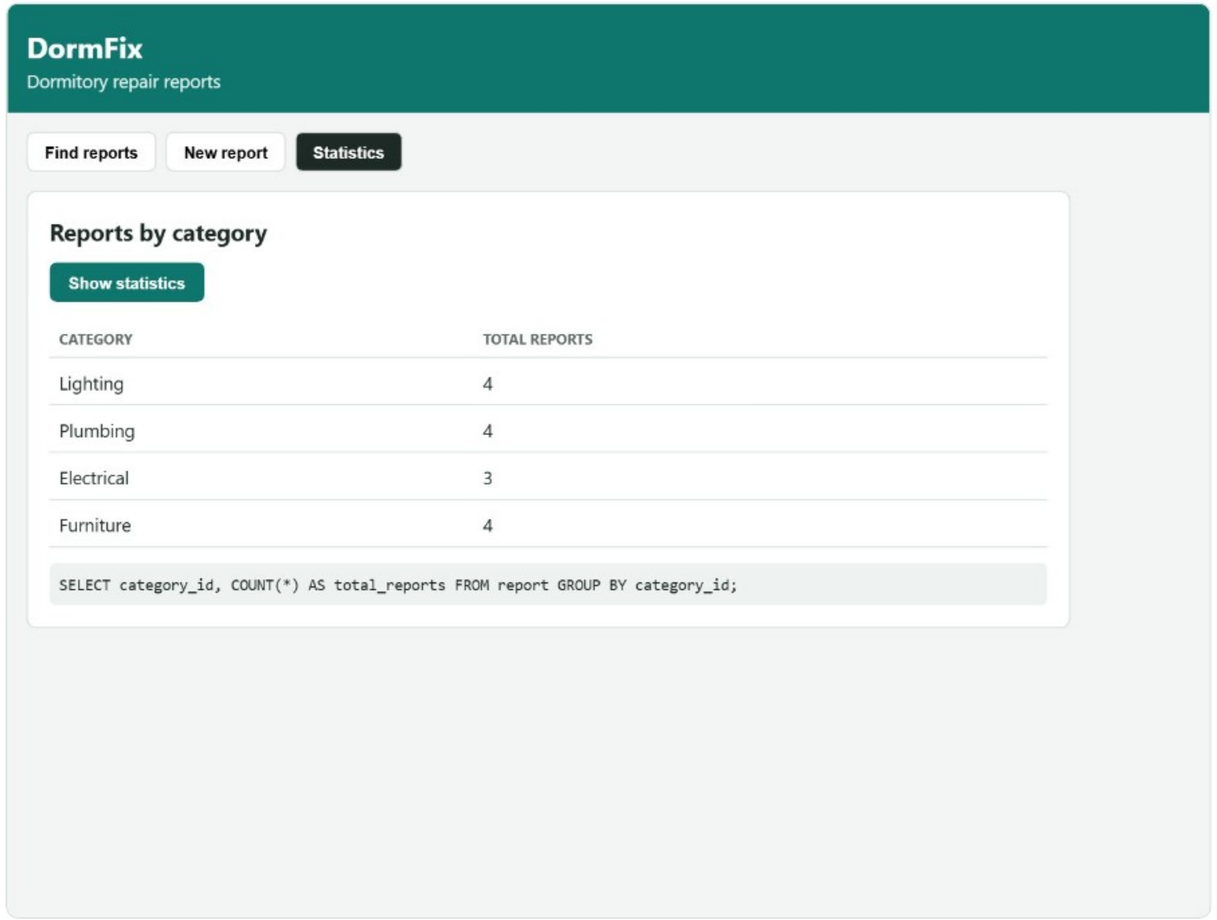

```
DELETE FROM report
WHERE report_id = 'DF-1016';
```

Table: report

Result: the row DF-1016 is removed. The table has 15 reports again.

User sees: “Report DF-1016 deleted.” The report disappears from the list.

Button “Show statistics” → COUNT + GROUP BY




```
SELECT category_id, COUNT(*) AS total_reports
FROM report
GROUP BY category_id;
```

Table: report

Result: Lighting 4, Plumbing 4, Electrical 3, Furniture 4 (15 in total).

User sees: a table with the number of reports for each category.

Working prototype

https://shynstudy.lol/dormfix/

Database: Supabase project “Dormfix” (PostgreSQL)

AI tools: Claude was used to generate the app code. We tested every button and checked each change in Supabase.

Thank you! We are ready for your questions.